Bloco · Algoritmos · Big-O (12 questões)

> Questões autorais no estilo FGV, elaboradas para treino. Não são questões oficiais.

> Assunto único: complexidade de algoritmos (Big-O). Modelo Tribunal do guia FGV: cenário com analista e sistema fictício, leitura de código em Python e Java, uma questão de afirmativas e uma de "respectivamente". Marque "?" no que não souber e nunca deixe em branco.

Questão 1

O analista Fábio revisa a função a seguir, do sistema e-CUSTAS, que consolida os valores de uma lista com n processos e, para cada processo, os valores dos 12 meses do ano:

```
def consolida(processos):
    n = len(processos)
    total = 0
    for p in processos:
        total += p.valor
    for i in range(n):
        for mes in range(12):
            total += valor_mensal(processos[i], mes)   # custo constante
    return total
```

A complexidade de tempo da função consolida, em função de n, é:

(A) O(2^n);

(B) O(n^2);

(C) O(n log n);

(D) O(n);

(E) O(log n).

Questão 2

A analista Patrícia encontrou a rotina a seguir no sistema SIGEP-PB, escrita em Java, que reprocessa um lote com n registros:

```
for (int i = 1; i < n; i = i * 2) {
    for (int j = 0; j < n; j++) {
        lote[j] = lote[j] + i;
    }
}
```

A complexidade de tempo da rotina é:

(A) O(log n);

(B) O(n);

(C) O(n log n);

(D) $O(n^2)$;

(E) $O(n^2 \log n)$.

Questão 3

A rotina de geração do relatório de produtividade do sistema e-METAS tem tempo de execução $\Theta(n^2)$, em que n é o número de processos analisados. Em testes, a rotina levou 4 segundos para processar 10.000 processos.

Mantidas as demais condições, o tempo estimado para processar 30.000 processos é:

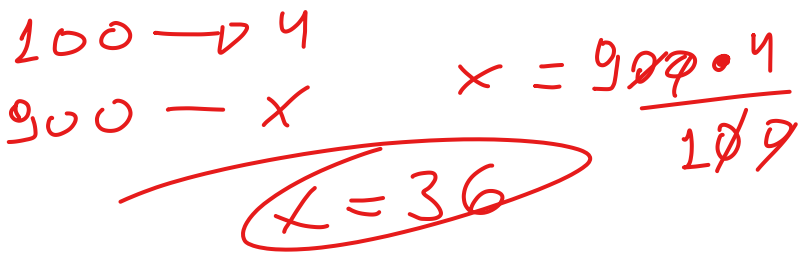
(A) 108 segundos;

(B) 36 segundos;

(C) 24 segundos;

(D) 16 segundos;

(E) 12 segundos.



Questão 4

O sistema e-CONSULTA mantém em memória um vetor com 2 milhões de números de processos, ordenado e sem alterações durante o expediente. O sistema recebe milhares de consultas por segundo para verificar se um número existe no vetor. A equipe do analista Henrique precisa da estratégia com o menor tempo de pior caso, sem usar memória adicional.

A estratégia mais adequada e a sua complexidade de pior caso são:

(A) busca binária, com complexidade O(log n);

(B) busca linear, com complexidade O(n), já que o vetor ordenado permite parar ao encontrar o número;

(C) tabela hash montada a partir do vetor, com complexidade O(1) no pior caso;

(D) busca binária, com complexidade O(n log n), pois o vetor precisa ser reordenado a cada consulta;

(E) busca linear, com complexidade O(log n), pois o vetor está ordenado.

Questão 5

A analista Simone encontrou, no módulo de cálculos do sistema e-DEPÓSITO, a função recursiva a seguir, chamada inicialmente com n igual ao tamanho do vetor v:

```
def soma_prefixos(v, n):
    if n == 0:
        return 0
    parcial = 0
    for i in range(n):
        parcial += v[i]
    return parcial + soma_prefixos(v, n - 1)
```

A complexidade de tempo da função soma_prefixos é:

(A) $O(2^n)$;

(B) $O(n^2)$;

(C) O(n log n);

(D) O(n);

(E) O(log n).

Questão 6

Em um treinamento sobre análise de algoritmos para a equipe de desenvolvimento de um Tribunal de Justiça, foram apresentadas as afirmativas a seguir.

I. Se o número de operações de um algoritmo é $f(n) = 3n^2 + 50n + 1000$, então $f(n)$ é $O(n^2)$.

II. Se um algoritmo é $O(n^2)$, então ele também é $O(n^3)$.

III. A notação Ω (ômega) descreve um limite superior assintótico para o tempo de execução de um algoritmo.

Está correto o que se afirma em

(A) I, apenas.

(B) III, apenas.

(C) I e II, apenas.

(D) II e III, apenas.

(E) I, II e III.

Questão 7

Para impedir o cadastro de guias com número repetido, o analista Diego escreveu duas versões de uma função para o sistema e-GUIAS, que recebe uma lista com n números:

```
def tem_duplicado_v1(numeros):
    for i in range(len(numeros)):
        for j in range(i + 1, len(numeros)):
            if numeros[i] == numeros[j]:
                return True
    return False

def tem_duplicado_v2(numeros):
    vistos = set()            # conjunto implementado com tabela hash
    for x in numeros:
        if x in vistos:
            return True
        vistos.add(x)
    return False
```

Considerando o pior caso das funções e o custo médio das operações sobre o conjunto, as complexidades de tempo de tem_duplicado_v1 e tem_duplicado_v2 são, respectivamente:

(A) O(n) e $O(n^2)$;

(B) $O(n^2)$ e $O(n^2)$, pois as duas versões percorrem a lista inteira;

(C) O(n log n) e O(n);

(D) $O(n^2)$ e O(1);

(E) $O(n^2)$ e O(n).

Questão 8

O sistema e-DISTRIBUI, de um Tribunal de Justiça, conta quantos processos pertencem a cada vara. O trecho em Java a seguir recebe uma lista com m varas e uma lista com n processos:

```
for (Vara v : varas) {
    for (Processo p : processos) {
        if (p.getVaraId() == v.getId()) {
            contagem[v.getId()]++;
        }
    }
}
for (Vara v : varas) {
    imprime(v, contagem[v.getId()]);
}
```

A complexidade de tempo do trecho é:

(A) O(n);

(B) O(m + n);

(C) $O(n^2)$;

(D) O(m · n);

(E) O(m).

Questão 9

A analista Lúcia analisa o método a seguir, do sistema e-PROTOCOLO, que procura um número de protocolo num vetor não ordenado com n posições:

```
public int localiza(String[] numeros, String alvo) {
    for (int i = 0; i < numeros.length; i++) {
        if (numeros[i].equals(alvo)) {
            return i;
        }
    }
    return -1;
}
```

As complexidades de tempo do método no melhor caso e no pior caso são, respectivamente:

(A) O(n) e O(n);

(B) O(1) e O(1);

(C) O(log n) e O(n);

(D) O(n) e O(1);

(E) O(1) e O(n).

Questão 10

O sistema e-QUITAÇÃO verifica se existem duas guias, num vetor de n valores ordenado em ordem crescente, cuja soma é exatamente o valor devido. O analista Marcelo implementou a função a seguir:

```
def existe_par(v, alvo):
    i, j = 0, len(v) - 1
    while i < j:
        soma = v[i] + v[j]
        if soma == alvo:
            return True
        elif soma < alvo:
            i += 1
```

fibbonati ?

```
        else:
            j -= 1
    return False
```

A complexidade de tempo da função, no pior caso, é:

(A) O(log n);

(B) O(n);

(C) O(n log n);

(D) $O(n^2)$;

(E) $O(2^n)$.

Questão 11

Dois fornecedores apresentaram ao Tribunal algoritmos para ordenar a fila de distribuição de processos. Para uma entrada com n processos, o algoritmo X executa 100n operações e o algoritmo Y executa n^2 operações.

Sobre os dois algoritmos, é correto afirmar que:

(A) para entradas com menos de 100 processos, Y executa menos operações, mas, acima de 100, X passa a ser mais eficiente, pois X é O(n) e Y é $O(n^2)$;

(B) X é mais eficiente para qualquer tamanho de entrada, pois tem complexidade linear;

(C) Y é mais eficiente para qualquer tamanho de entrada, pois não tem constante multiplicativa;

(D) X e Y têm a mesma complexidade assintótica, já que executam o mesmo número de operações quando n = 100;

(E) X é O(100n) e, por isso, é menos eficiente do que Y para entradas grandes.

Questão 12

O analista Anderson encontrou, no sistema e-ARQUIVO, a função recursiva a seguir, que organiza um vetor v com n documentos. A função junta percorre uma única vez os elementos de esq e dir, com custo proporcional ao tamanho dos dois juntos:

```
def organiza(v):
    if len(v) <= 1:
        return v
    meio = len(v) // 2
    esq = organiza(v[:meio])
    dir = organiza(v[meio:])
    return junta(esq, dir)
```

A complexidade de tempo da função organiza é:

(A) O(log n);

(B) O(√n);

(C) O(n);

(D) O(n log n);

(E) $O(n^2)$.

Minhas respostas

Questão	Resposta
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	